

pkgsentinel

발표 대본 + 시스템 설명서 (QnA 참조용)

7팀 캡스톤 · 2026-05-31

Part 1. 발표 대본(출처 표기) | Part 2. 시스템 설명서

pkgsentinel 발표 대본 (출처 표기판)

괄호 안 = "어떤 자료 참고했어요?" 대비 인라인 출처. 전체 목록은 맨 아래 [참고문헌].

[슬라이드 1]

안녕하세요, 7팀 캡스톤 '패키지센티널' 발표 시작하겠습니다. 발표에 앞서, 원래 저희 주제가 AI 슬롭스쿼팅 탐지 모듈이었는데, 범위가 좁고 부족해 보여 도중에 무산시키고 주제를 바꿔 처음부터 다시 5주간 진행했습니다. 보고서에 다른 점이 좀 있을 수 있습니다.

최근 AI가 코드를 생성·실행하면서 두 갈래 위험이 생겼습니다.

첫째는 슬롭스쿼팅입니다. LLM이 존재하지 않는 패키지명을 환각하고, 공격자가 그 이름을 선점합니다 (슬롭스쿼팅 개념·환각을 선행연구: Spracklen et al., "We Have a Package for You: ... Package Hallucinations by Code Generating LLMs", USENIX Security 2024). 저희가 gpt-5.2-claude-opus-4-8로 1,000회 직접 측정했더니 (자체 실험, 2026-05; OpenAI-Anthropic API), 추출 패키지의 3.1%가 레지스트리에 없는 환각 이름이었습니다 (검증: PyPI·npm 레지스트리 API). 중요한 점은 환각이 예측 가능하다는 것입니다 — 동일 시퀀스 질문에서 5회 중 3회 이상 같은 환각을 보인 사례가 1/3이었습니다 (자체 반복성 측정). 이는 공격자가 패키지명을 선점할 수 있다는 의미입니다.

둘째는 agentic 패키지입니다. MCP 서버가 출시 1.5년 만에 9,400개 이상 폭증했고 (MCP 생태계 통계: DigitalApplied "MCP Ecosystem H1 2026", PulseMCP·Smithery·공식 MCP Registry 집계), npm 안에 'mcp' 키워드 패키지만 49,000개입니다 (npm registry 검색 API, 2026-05). 그 안에 자격증명 탈취를 비롯한 악성 패턴이 등장했습니다 (DataDog ai-skills 데이터셋).

[슬라이드 2~3]

해결책은 두 가지입니다. 탐지 엔진으로 지금 등록된 악성을 잡고, 제작자가 권한을 선언하게 해 위험을 구조적으로 줄이는 표준 제안입니다.

[슬라이드 4 — 구조]

전체 구조입니다 (다이어그램: C4 model — Simon Brown, c4model.com). 개발자·CI/CD가 CLI나 API로 패키지를 검사하면, 분석 엔진이 아카이브를 설치 없이 메모리로 스트리밍해 분석합니다. 외부로는 PyPI/npm 레지스트리, OSV 인텔 (Google OSV), MITRE ATT&CK (attack.mitre.org), 그리고 코드 의미·버전 변경·선언 의존성을 각각 보는 3-에이전트 Claude API (Anthropic Claude API)와 프로토콜 단위로 연동하고, 결과는 SBOM·STIX·SIEM 표준 포맷으로 출력합니다.

[슬라이드 5 — 7계층]

탐지를 7계층으로 구성했습니다. 정적 분석, MITRE ATT&CK TTP 매칭 (행위→ATT&CK 매핑 선행: Malware2ATT&CK, ScienceDirect 2024; 임베딩: Sentence-BERT all-MiniLM-L6-v2, Reimers & Gurevych 2019), 47개 악성 지표 ("Unveiling Malicious Logic..." arXiv:2512.12559, 2025), 테인트 분석 (Taint-Based Slicing 기법), LLM 검증을 하나의 파이프라인으로 통합했습니다.

[슬라이드 6 — Manifest]

제작자가 파일·네트워크·셸 등을 매니페스트에 선언하면, 도구가 실제 탐지 행위와 대조해 미선언 권한을 경고합니다. 선언-탐지 대조 자체는 Android의 'permission gap' 연구에서 잘 검증된 패러다임입니다 (Felt et al., "Android Permissions Demystified" / Stowaway, ACM CCS 2011). 모바일에서 검증된 이 패러다임을 신흥 agentic 패키지/스킬 영역으로 가져오고, 다중 어휘 정규화로 MITRE TTP 탐지 결과와 연결했습니다 (선언형 권한 선례: Anthropic MCP 권한 모델; Gilbertson, npm 권한 제안 2018; mcp-sec-audit; 어휘는 Deno permissions-Google Capslock 차용).

[슬라이드 7 — npm 비교]

실제 데이터로 검증했습니다. DataDog 악성 패키지와 인기 정상 패키지를 조합해 npm 273개 데이터셋을 검사했습니다 (악성: DataDog/malicious-software-packages-dataset, 시드 고정 무작위 추출; 정상: npm 인기 패키지). DB를

조회하지 않은 행위 기반만으로도 오탐이 없었습니다. 같은 데이터에서 Socket과 대등 이상입니다 ([비교: Socket.dev, OSV-Scanner](#)).

[슬라이드 8 — PyPI 비교]

PyPI에서도 검증했습니다. 표준 Python SAST인 Bandit ([PyCQA](#)), 다언어 SAST인 Semgrep ([Semgrep Inc](#), [Docker 이미지](#))과 비교했습니다. 범용 SAST의 한계가 보입니다. 또한 행위 기반만이면 오탐이 나오지만 LLM 교정으로 오탐이 2개까지 줄었습니다.

[슬라이드 9 — zero-day]

3개의 독립 명명 캠페인을 zero-day 환경에서 검증했습니다 — 2025-09 Shai-Hulud, 3주 전 Mini Shai-Hulud ([TeamPCP; Orca Security·The Hacker News 보도, 2026-05](#)), 8일 전 TrapDoor ([The Hacker News 보도, 2026-05](#)). 현재 구할 수 있는 패키지로 데이터셋을 구성했습니다 ([DataDog 데이터셋](#)). npm의 Bandit 격인 njsscan과 비교했고 ([njsscan, MobSF/OpenSecurity](#)), OSV 0%는 DB 영향이 없는 zero-day를 가정한 지표입니다 ([Google OSV](#)). 페이로드 없는 decoy 6개를 제외하면 90%입니다.

[슬라이드 10 — 남은 과제 ai-skills]

남은 과제입니다. DataDog 데이터셋 중 최근 ai-skills 패키지셋은 결이 다른 zero-day입니다 ([DataDog ai-skills 카테고리, 2026-05 신설](#)). 공격이 코드가 아닌 markdown 지시문 자체에 있습니다. 지시문 분석 모듈 markdown-pinpoint를 만들어 넣었더니 탐지율이 소폭 상승했습니다 ([프롬프트 인젝션 패턴은 Sentry skill-scanner\(getsentry/skills\)에서 적응; 스킬 공격 선행: Snyk ToxicSkills·SkillRisk·Invariant Labs tool-poisoning·OWASP MCP Top 10](#)). 이처럼 권한을 선언시켜 매니페스트로 만들면 미선언 행위 탐지에 긍정적입니다.

발표는 여기까지입니다. 감사합니다.

참고문헌 (References)

선행연구 (학술)

- Spracklen et al., [We Have a Package for You: A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs](#), USENIX Security 2024 — 슬롭스쿼팅/환각률
- Felt, Chin, Hanna, Song, Wagner, [Android Permissions Demystified](#) (Stowaway), ACM CCS 2011 — 선언↔탐지 permission gap
- [Malware2ATT&CK](#) (ScienceDirect S0167404824000737, 2024) — 행위→ATT&CK 매핑
- [Unveiling Malicious Logic...](#) arXiv:2512.12559 (2025) — 47-indicator
- Reimers & Gurevych, [Sentence-BERT](#) (all-MiniLM-L6-v2), 2019 — TTP 임베딩
- mcp-sec-audit / Gilbertson npm 권한 제안(2018) — 매니페스트 선례

표준·프레임워크

- MITRE ATT&CK ([attack.mitre.org](#)) · OSV (Google) · C4 model (Simon Brown) · OWASP MCP Top 10 · Anthropic MCP

비교 도구

- Socket.dev · OSV-Scanner(Google) · Bandit(PyCQA) · Semgrep(Semgrep Inc) · njsscan(MobSF) · GuardDog(DataDog, overfit으로 비교 제외)
- 스킬 스캐너: Sentry skill-scanner(getsentry/skills) · SkillRisk · Snyk Agent Scan/ToxicSkills · Invariant Labs(tool poisoning)

데이터셋·인프라

- DataDog/malicious-software-packages-dataset (악성 코퍼스; npm·PyPI·ai-skills)
- npm·PyPI 레지스트리 API · MCP 생태계 통계(DigitalApplied/PulseMCP/Smithery)
- Anthropic Claude API(GPT-5.2·Claude Opus 4.8 환각실험; Sonnet 4.5 검증 에이전트) · SQLCipher(Zetetic)
- 어휘 차용: Deno permissions · Google Capslock

사건(검증 대상)

- Shai-Hulud(2025-09) · Mini Shai-Hulud(TeamPCP, 2026-05; Orca Security·The Hacker News) ·
TrapDoor(2026-05; The Hacker News)

pkgsentinel 시스템 설명서 (QnA 참조용)

발표 중 질답 시 옆에 두고 보는 레퍼런스. 대본은 앞장 참조.
톤 원칙: "발명이 아니라 현실 위협의 깊은 이해 + 동작하는 통합 시스템 + 실제 멀웨어/표준도구로 정직한 측정 + 한계 발견·개선". 선행 도구 인정 = 약점이 아니라 강점.

0. 한 문장 정의

pkgsentinel = PyPI·npm에 등록되어 있지만 악의적인 패키지를, 설치 없이 아카이브를 메모리로 스트리밍해 7계층/21단계로 분석하는 공급망 보안 엔진 + 제작자가 권한을 선언하게 해 위협을 구조적으로 줄이는 **Agentic Capability Manifest** 표준 제안.

핵심 위협 모델 2갈래:

- 1. 슬롭스쿼팅 — LLM이 없는 패키지명을 환각 → 공격자가 그 이름 선점 → 개발자가 AI 추천 믿고 설치.
- 2. Agentic 패키지/스킬 — MCP 서버·AI 스킬이 폭증하며 코드가 아닌 자연어 지시문(SKILL.md)에 악성이 숨음.

1. 핵심 용어집 (교수님 질문 대비)

용어	한 줄 설명
슬롭스쿼팅 (slopsquatting)	타이포스쿼팅의 AI판. 오타가 아니라 LLM 환각 패키지명 을 공격자가 선점. (USENIX Security 2024, Spracklen et al.)
TTP	Tactics·Techniques·Procedures. 공격자의 행위 패턴. MITRE ATT&CK 가 업계 표준 분류.
MITRE ATT&CK	실제 공격 행위를 정리한 공개 지식베이스(attack.mitre.org). TTP는 MITRE 표준이고, 우리 기여는 그걸 임베딩해 패키지 행위와 의미적으로 매칭한 부분 .
SAST	Static Application Security Testing. 코드를 실행하지 않고 정적 분석해 취약점 탐지. Bandit·Semgrep·njsscan이 대표.
DAST / SCA	DAST=실행 중 분석, SCA=의존성 구성요소 분석(알려진 취약 버전 매칭).
F1 score	precision과 recall의 조화평균 = 2·P·R/(P+R). 둘 중 하나만 높은 걸 못 속이게 하는 균형 지표. 분류 성능의 표준 단일 수치 — 교수님들이 익숙.
precision(정밀도)	악성이라 한 것 중 진짜 악성 비율. 낮으면 오탐(false positive) 많음.
recall(재현율)	진짜 악성 중 잡아낸 비율. 낮으면 미탐(false negative) 많음.
zero-day	공개·패치 전이라 DB·시그니처에 아직 없는 신규 위협. DB 기반 탐지는 정의상 0%.
agentic 패키지	AI 에이전트가 쓰는 도구/스킬 패키지(MCP 서버, Claude Skill 등). 파일·셀·네트워크 권한을 갖고 자율 행동.
MCP	Model Context Protocol(Anthropic). LLM이 외부 도구·데이터에 붙는 표준. 출시 1.5년 만에 서버 ~9,400개로 폭증.
taint analysis(테인트 분석)	민감 source(env/파일/비밀)에서 위험 sink(http/exec)로 데이터가 흐르는지 추적.
install-hook	npm package.json의 postinstall 등 설치 시 자동 실행 스크립트. 악성 송출의 단골 통로.
homoglyph	사람 눈에 같아 보이는 글자(0↔O, l↔I↔1)로 만든 위장 패키지명.

2. 7계층 ↔ 21단계 전체 맵

탐지는 7계층(L0~L6) 으로 묶고, 실제 실행은 21단계(stage_00~20) 로 순차 수행. 필수 단계(★)는 하나라도 실패하면 `verdict=ERROR` (부분 판정 금지).

Layer 0 — 레지스트리 / 위협 인텔 (게이트)

- **stage_00 레지스트리 확인** — PyPI/npm에 존재하는가. 미등록 → `CANNOT_ANALYZE` (소스 없음).
- **stage_01 Threat Filter** — 암호화 DB(SQLCipher)에서 known_malicious/popular/typosquat 매칭. 알려진 악성 exact match → 즉시 `MALICIOUS` 단축. (220k+ OSV/GHSA + OSSF malicious-packages)
- **stage_02 공격 이력(OSV)** — 이름·버전 기준 과거 사고 조회. 현재 버전이 affected에 없으면 historical 로 분리.
- **stage_03 OpenSSF Scorecard / stage_04 SLSA 프로비넌스** — 참고 메타(판정 직접 영향 X).

⚠ **측정 시 Layer 0(특히 stage_01 OSV 조회)는 끔.** eval은 `Layer 0 OFF` = 순수 행위 분석 성능을 본 것. zero-day의 "OSV 0%"도 정의상(공개 전 DB에 없음) 0이지 측정 실패가 아님.

Layer 1 — 소스 추출 + Agentic 게이트

- **stage_05 캐시 조회** — 6-트리거 무효화(아카이브 해시 변하면 재분석).
- **stage_06 전 파일 소스 추출 ★기반** — 아카이브를 설치 없이 메모리로 스트리밍, 핵심 파일만 추출.
- **stage_07 Agentic 분류** — MCP/스킬 패키지인지 판별. agentic + 악성이면 단축, 아니면 일반 파이프라인으로.

Layer 2 — 행위 시퀀스

- **stage_08 Behavior Sequence ★필수** — Python AST + JS tree-sitter로 API 호출을 4차원(파일/네트워크/실행/은닉)으로 추출.

Layer 3 — 패턴 매칭 (탐지의 본체)

- **stage_09 문자열 상수 풀** — base64/hex/고엔트로피 난독화 문자열.
- **stage_10 버전 diff** — 이전 버전 대비 갑자기 추가된 위험 API(event-stream.xz류 대응).
- **stage_11 TTP 매칭 ★필수** — MITRE ATT&CK TTP를 임베딩(Sentence-BERT all-MiniLM-L6-v2) 해 행위와 의미 유사도 매칭. 룰 정확매칭이 아니라 **룰 없는 변종도 의미로 잡음**.
- **stage_12 카테고리 이상** — 패키지 설명 vs 실제 행위 불일치.
- **stage_13 47-indicator 매치** — 논문(arXiv:2512.12559) 기반 악성 지표 분류(install-hook, exec/eval, credential theft 등). 파일 단위 risk-combo 승격.
- **stage_14 Sequential pattern mining** — 호출 시퀀스 패턴.
- **stage_15 Taint slicing** — source→sink 흐름만 추출해 LLM 프롬프트 토큰 절약.

Layer 4 — LLM 이중 검증

- **stage_16 LLM 검증 ★필수** — Claude 멀티 에이전트 3종(의미/버전-diff/의존성)이 각자 판정 후 **합의 투표(consensus)**. 정상 패키지 오탐을 강등하는 정밀도의 핵심(행위만이면 FP 45 → LLM 후 2). stub 모드=결정적 룰(키 없을 때).
- 합의 규칙: 2+ MALICIOUS→MAL / 1 MAL+1+ SUS→MAL / 2+ SUS→SUS / 그 외 BENIGN.

Layer 5 — 추가 분석 (옵션)

- **stage_17** 의존성 재귀(--deps) · **stage_18** 바이너리(.so/.dll/.node) · **stage_19** 샌드박스(--sandbox).

Layer 6 — 판정 + 표준 출력

- **stage_20** NIST SSDF 준수 + 최종 verdict 결정.
- 출력 표준: CycloneDX 1.5 VEX, STIX 2.1/TAXII, HMAC 서명 웹훅, Falco/Tetragon 룰, SafeDep pmg.

3. 판정 규칙 (verdict_rules.py)

오직 **Evidence** + **stage** 결과로만 판정. 패키지 나이·인기도·다운로드 수는 일절 안 봄(편향 방지).

verdict	조건
MALICIOUS	high-severity TTP ≥ 1 AND LLM=malicious AND 최강 악성 evidence confidence ≥ 0.85
HIGH_RISK	(TTP match ≥ 1 OR version_diff critical) AND LLM $\in$ {suspicious,malicious}
SUSPICIOUS	weak TTP(LLM $\neq$ BENIGN) OR version_diff any OR LLM-only suspicious(conf ≥ 0.5 , quorum 2)
CLEAN	모든 단계 통과 AND evidence 없음 / 또는 weak TTP+LLM=BENIGN (BENIGN 이 약신호를 덮어 씌)
ERROR	필수 단계(08/11/16) 중 하나라도 실패
CANNOT_ANALYZE	stage_00 미등록 확정

핵심 설계: LLM BENIGN 판정이 약한 TTP 신호(유사도 0.70~0.85, severity LOW)를 덮어씌 → 정상 패키지 오탐 억제. 강매칭(≥ 0.85 , severity $\neq$ LOW)은 LLM과 무관하게 위험 분기.

4. 데이터셋 & 측정 방법론

DataDog malicious-software-packages-dataset

- 규모(2026-05): 악성 아카이브 ~22,052 / 고유 패키지 ~11,704. (npm malicious_intent 20,588 + compromised_lib 1,260 + ai-skills 204; PyPI malicious_intent 1,806 pkg / 2,483 zip 별도; ide_extensions 신설 0)
- 성격: 사고 후(post-discovery) 스냅샷. 파일명 날짜=발견일. 사람이 수동 triage.
- README가 직접 인정: "Most ... identified by GuardDog" + "selection bias ... single ruleset (GuardDog)".

샘플링 (체리피킹 아님)

- `_pick_datadog_samples(echo, kind, limit, seed=42)`: 고정 시드 42 → shuffle → 상위 추출. 재현 가능. >10MB 제외.
- 정상(benign)은 큐레이션된 현재 인기 패키지(npm: react·vue·angular... / PyPI: requests·urllib3·fastapi...). → 실제 레지스트리 패키지로 오탐 검증.
- ai-skills는 전수 204개. zero-day는 일부러 OSV 비등록 부분집합(더 어려운 쪽) 선택.

5. 측정 결과 총괄 (전부 Layer 0 OFF, 명시 없으면)

(A) npm 비교 — n=273 (Socket 분석가능 동일 분모)

도구	recall	precision	F1
pkgsentinel (L0 OFF, 행위만)	0.652	1.000 (FP 0)	—
pkgsentinel (L0 ON)	0.912	—	—
pkgsentinel (프로덕션)	—	—	0.974
Socket.dev	0.937	—	0.962
- 메시지: 행위만으로도 오탐 0 , \$1B 유니콘 Socket와 동급 이상.			

(B) PyPI 비교 — n=139

도구	recall	precision	F1
pkgsentinel (claude)	0.886	0.972	0.927
pkgsentinel (stub)	0.886	0.843	—
Bandit (PyCQA)	0.487	0.470	—
Semgrep (Docker)	0.212	0.298	—
OSV	0.76 (사후 등재)	—	—
- 메시지: 범용 SAST는 '악성 분류기'가 아니라 코드이슈 탐지기 → 정상도 대량 flag. LLM 검증이 정밀도 핵심(FP 45→2).			

(C) zero-day 다중 사건

사건	pkgsentinel	OSV	njsscan
Shai-Hulud (2025-09)	100%	0%	0%
Mini Shai-Hulud (2026-05)	100%	0%	0%
TrapDoor (2026-05)	78% (decoy 6 제외 시 90%)	0%	0%
- OSV 0%는 공개 전 정의상 0(미측정 아님). TrapDoor 분석 중 install-hook 네트워크 송출 미탐 갭 발견 → 직접 구현, 61%→78%(정상 FP 0).			

(D) ai-skills — n=204 (동일 분모, 정직한 한계)

도구	recall
OSV-Scanner	0.000
Semgrep	0.098
Bandit	0.142
pkgsentinel	0.152
+ markdown-pinpoint(프로토타입)	0.284
- 전 도구가 실패하는 open problem. 이유: 악성이 코드가 아니라 SKILL.md 자연어 지시문, 실행 엔진이 AI 에이전트 자신. 204개 중 77%(158개)가 Python 코드 없음 → SAST 구조적 실명.	

(E) 환각률 (최신 모델 gpt-5.2-claude-opus-4-8, 1,000회)

- raw 환각 6.65% → 검증 후 완전허구 3.13%(A등급 1.10%).
- 반복성 32.6%: 동일 질문 5회 중 3회+ 같은 환각 → 예측 가능 = 공격자 선점 가능.

6. Agentic Capability Manifest (가장 연구적인 부분)

- 개념: 제작자가 필요 권한(파일·네트워크·셸 등)을 매니페스트에 선언 → 도구가 실제 탐지 행위와 대조 → 미선언 권한 경고.
- 선례(EXTEND, 대체 아님): Android **permission gap** 연구(Felt et al./Stowaway, CCS 2011)가 검증한 패러다임. 선언형 권한은 MCP 권한모델·Gilbertson(npm 2018)·Deno permissions·Google Capslock에 선례. 우리는 이를 **패키지/스킬 영역으로 확장**.
- **CANONICAL(정규 어휘)**: 도구마다 권한 표현이 제각각(self 27종, MITRE 28종) → **15개 표준 capability**로 정규화(filesystem-read/write, shell, network, env-secrets, code-exec, credential-paths, db-access, mcp-client/server, llm-call, tool-loop, memory-persistent, dynamic-tool-load, agent-to-agent).
`cluster.normalize_set()` 가 선언·탐지를 같은 어휘로 맞춰 비교.

7. 비교 도구 (각각 무엇인가)

도구	만든 곳	종류	비고
Socket.dev	Socket Inc (\$60M Series C, \$1B 유니콘, OpenAI Trusted Access)	행위 기반 공급망	가장 강한 상용 비교군
OSV-Scanner	Google	DB 매칭	zero-day엔 정의상 0
Bandit	PyCQA	Python 전용 SAST	표준 Python SAST
Semgrep	Semgrep Inc	다언어 SAST	Windows 네이티브 불가 → Docker
njsscan	MobSF/OpenSecurity	Node.js SAST	npm의 Bandit 격
GuardDog	DataDog	룰+YARA+메타 SAST	비교 제외 — 데이터셋 식별자라 overfit/circular

8. 정직성·한계 (먼저 말하면 강점)

1. **GuardDog 양날의 검**: 데이터셋이 GuardDog-findable 악성에 치우침 → 우리 0.886도 그 위 수치. "**GuardDog보다 recall 높다**" 주장 안 함. 차별은 LLM 검증/TTP 임베딩/Manifest/스킬지시문 차원. (저장: guarddog_dataset_bias.md)
2. **decoy**: TrapDoor 6개는 페이로드 없는 미끼 → 제외해도 90%지 **100% 아님**. 4개 진짜 미탐 남음.
3. **SAST 비교 한계**: "단순 SAST 이기는 건 당연" 지적 → 인정. SAST 비교는 '**우리가 이긴다**'가 아니라 '**범용 SAST로는 악성 분류가 안 된다**'는 갭 동기 부여. 진짜 승부는 통합·LLM·Manifest.
4. **ai-skills 0.15**: 기술적 미탐 맞음. 단 전 도구 실패 open problem이고 프로토타입으로 2배 개선.
5. **markdown-pinpoint**: 프로토타입(미통합) → 별도 막대로 표기. 패턴은 Sentry skill-scanner 적용 (docstring 명시).
6. **환각**: raw는 구·신 모델 비슷(~6.7%). 위험은 모델 세대 무관 — 1/3 예측가능 반복이 근거.

9. 예상 까다로운 질문 → 답 (요약)

- Q. 선행 도구 있는데 차별점? → 최초 주장 안 함. 통합이 차별(코드+install-hook+스킬지시문+manifest를 한 도구). \$1B 유니콘·Google 도구와 같은 벤치마크 정직 측정.
- Q. 데이터 출처가 DataDog인데 GuardDog 쓰면 되잖아? → GuardDog가 그 데이터 식별자 → 자기 결과 재채점=circular. 독립 도구로 비교한 게 공정성.
- Q. 7계층/TTP 너희 발명? → 아니오. TTP=MITRE 표준, 매핑도 Malware2ATT&CK 선례. 임베딩·매칭 엔진을 패키지에 구현·측정한 게 우리 부분.
- Q. zero-day인데 OSV 76%? → 사후 등재율. 공격 순간엔 정의상 0. DB는 항상 한발 늦음.
- Q. 우리도 SAST 아니냐? → 정적 분석 쓰니 넓은 의미 SAST 맞음. 단 범용 코드이슈 SAST와 달리 악성 분류 전용 + 의미 계층(LLM/TTP) + 선언형 Manifest가 차별.
- Q. ai-skills 0.15 실패 아닌가? → 네 미탐 인정. 단 전 도구 실패 open problem, 프로토타입 2배 개선 + Manifest 경로 제시.
- Q. 최신 모델인데 환각 왜 안 사라지나? → raw 비슷(~6.7%). 1/3 예측가능 반복이라 모델 세대 무관하게 위협 유효.